

sem_6_paper__Copy_ (1) (1).pdf

-  My Files
-  My Files
-  Chandigarh University

Document Details

Submission ID

trn:oid::3618:133532917

Submission Date

Mar 30, 2026, 1:00 PM GMT+5:30

Download Date

Mar 30, 2026, 1:04 PM GMT+5:30

File Name

sem_6_paper__Copy_ (1) (1).pdf

File Size

1.9 MB

6 Pages

3,781 Words

22,063 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



ML-Optimized Stochastic Vehicle Routing for Urban Delivery Services

1st Aryan Dhasmana 2nd Nikhil Gusain 3rd Vishal Gupta 4th Simranjit Kaur 5th Isha Aggarwal
 AIT CSE AIT CSE AIT CSE AIT CSE AIT CSE
 Chandigarh University Chandigarh University Chandigarh University Chandigarh University Chandigarh University
 Mohali, India Mohali, India Mohali, India Mohali, India Mohali, India
 23bcb10081@cuchd.in 23bcb10033@cuchd.in 23bcb10080@cuchd.in 23bcb10084@cuchd.in 23bcb10053@cuchd.in

6th Dr. Gurwinder Singh
 AIT CSE
 Chandigarh University
 Mohali, India
 gurwinder.e11253@cumail.in

Abstract—Last-mile delivery routing in cities is very hard to do because of two main sources of uncertainty: random demand arrivals and traffic conditions that change over time and affect travel times while the delivery is happening. This paper presents a practical replication and extension study of the SVRPBench stochastic vehicle routing framework, adapted and evaluated in a new urban context—Chandigarh, India. We create a complete city-wide pipeline that: (1) gets spatio-temporal order arrivals from open data sources, (2) learns demand and traffic prediction models (like ARIMA, LSTM, and decision-focused SPO+ approaches), and (3) combines these forecasts with optimisation and metaheuristic methods (like OR-Tools, ALNS, ACO, ABC, and nearest_neighbor heuristics) to make strong delivery routes. A scenario-based simulator tests how well a route holds up under real-world random changes, such as traffic jams, noise in travel times, and accidents. Swarm-based metaheuristics are consistently outperformed by improvement-based heuristics and classical solvers, according to experimental evaluation across representative instances. While ALNS shows better service quality with 12.34% less mean lateness than ABC, OR-Tools achieves the lowest mean cost per delivery (4.09% improvement over the lowest-performing solver). Lower-cost solutions are typically associated with better service performance, according to a moderately positive correlation ($r = 0.667$) between cost and lateness. Nearest_neighbor achieves about 65% less runtime than ALNS while maintaining competitive solution quality, according to computational efficiency analysis. For urban delivery networks, the study shows that deterministic and improvement-based routing strategies are still very competitive, especially when balancing cost effectiveness and service dependability in stochastic environments. An accessible platform for operational decision support is provided by a lightweight web prototype that displays routes, interruptions, and KPIs.

Index Terms—Stochastic Vehicle Routing, SDVRP, Last-Mile Delivery, Demand Forecasting, Traffic Prediction, Robust Optimization, Metaheuristics, Benchmarking

I. INTRODUCTION

Urban delivery has become a daily reality for households and businesses, but it remains operationally difficult because route plans degrade quickly once real traffic and new orders arrive. Verification of last-mile logistics declared that

routine demand uncertainty, and time dependent congestion are the prominent driving factors behind cost and services failure[6][20]. Meanwhile, the research community shifted purely beyond the deterministic VRP models, considering stochastic and dynamic VRP models where travel times and customers' requests are random processes [5][4]. SVRPBench has been developed as a benchmark and a library aimed at testing the robustness of vehicle routing approaches considering stochastic dynamics similar to those found in real urban environments[1]. Motivated by SVRPBench, our goal is to replicate the same evaluation philosophy on Chandigarh and then systematically test multiple routing paradigms—classical solvers, metaheuristics (including our previous bee/ant hybrids), and learning-augmented methods [1][8]. The city of Chandigarh is a good candidate since it is sufficiently dense to exhibit realistic congestion phenomena, but the sectoral planning also facilitates understanding clustering and demand patterns.

A. Contributions

This paper contributes a city-focused, end-to-end workflow:

- Chandigarh SVRP instance generation: A reproducible process for building road graphs and travel time profiles using open data sources [12].
- Forecasting models: machine learning baselines for demand intensity prediction and edge-zone travel time multipliers [7][8].
- Hybrid routing stack: a stack of robust deterministic approaches and efficient metaheuristic solvers, including OR-Tools as the reference, ALNS, ACO/ABC, and learning-augmented heuristics, compared under the same stochastic simulations [13][10][11].
- Evaluation and visualization: scenario-based simulation with service-level metrics (lateness, robustness, re-plan frequency), including a web prototype for route exploration and routing [1].

II. BACKGROUND AND RELATED WORK

A. Vehicle Routing under dynamic scenario

This has been a problem as long as the railways have existed alongside other medium of travels [2]. Multiple substitutions to regular approach including time window, stochastic travel times and scenarios alongside a varying demand [3][4]. The recent developments have been uneffectives under the dynamic nature of urban logistics which is full of chaos which is a rule more than exception [5].

B. Predict over Optimize using ML in VRP

A broad survey of ML for VRP has identified two main ways for learning: (i) predicting uncertain inputs and (ii) learning policies/heuristics that construct routes directly. [8] In a real system, typically predicting uncertainties and using them to optimize the model is done.[17] However, this does not ensure good decisions are being made in operation even if prediction errors are low.[16] The main idea behind SPO/SPO+ is that it is not just important to be good in making decisions in terms of accuracy but also in terms of how good the final decision is.[16][17].

C. Learning-Based Routing and Hybrid Methods

Deep reinforcement learning employs learned constructive policies for capacitated VRP and its variants, as shown in [10] and [9]. However, some of the RL methods are challenging to scale and enforce hard operational constraints, as shown in [11] and [8]. However, the use of a combination of learning and other search strategies, such as local search, neighborhood search, or decomposition, can improve results.

D. Metaheuristics in Delivery Routing

Metaheuristics (GA, ACO, ABC, SA, VNS, ALNS) are commonly used methodologies for solving large-scale VRPs because they give strong results along with cost-effectiveness [13][14]. In our previous study on bee-inspired algorithm for drone delivery, it gave competitive results under operation constraints [21]. Hence, this study will extend and evaluate similar algorithms for stochastic traffic and orders in SVRP [1]

III. PROBLEM SETTING

We model a delivery region (Chandigarh) as a directed graph $G = (V, E)$ derived from a road network. A depot is node 0, and customers are nodes $i \in \{1, \dots, n\}$ with demands q_i and service times s_i . Vehicles $k \in \{1, \dots, m\}$ have capacity Q_k .

A. Stochastic, Time-Dependent Travel Time

Travel time on edge $(u, v) \in E$ at time t is a random variable:

$$T_{uv}(t) = \tau_{uv}(t) \cdot \epsilon_{uv}(t), \quad (1)$$

where $\tau_{uv}(t)$ is the predicted mean (time-of-day dependent) and $\epsilon_{uv}(t)$ captures stochastic shocks (incidents, micro-congestion). This abstraction matches common SDVRP modeling choices [5][1].

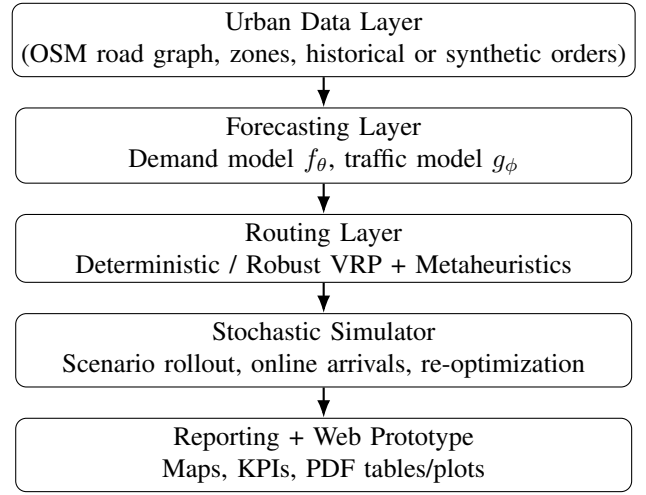


Fig. 1. Chandigarh SVRPBench-style pipeline: learn forecasts, optimize routes, and evaluate robustness via simulation. [1][12]

B. Demand Arrivals

Customer requests may be known upfront (static) or arrive online (dynamic). We represent arrivals by an intensity model (e.g., per-zone Poisson process):

$$\lambda_z(t) = f_\theta(\mathbf{x}_{z,t}), \quad (2)$$

where z denotes a spatial zone and $\mathbf{x}_{z,t}$ includes calendar and contextual features. Demand forecasting for routing is widely documented in last-mile analytics [6][7].

C. Objective

A common operational objective is minimizing expected cost plus service penalties:

$$\min_{\pi} \mathbb{E}[C(\pi, \omega)] + \beta \mathbb{E}[P(\pi, \omega)], \quad (3)$$

where π denotes the route plan (and possibly a recourse policy), ω is the stochastic realization (traffic, incidents, arrivals), C is travel+fleet cost, and P penalizes lateness or missed deliveries. Robustness can also be measured with risk terms (e.g., CVaR) [18][19].

IV. METHODOLOGY

Our methodology follows the SVRPBench philosophy: *generate/collect stochastic dynamics, plan routes, then simulate execution under uncertainty* to evaluate robustness [1].

A. System Pipeline

Fig. 1 summarizes the end-to-end workflow.

B. Data Construction for Chandigarh

Road network: We extract Chandigarh's drivable street network using OSMnx and build a weighted directed graph [12].

Zones and demand points: Customer locations are sampled per sector/zone to support interpretable spatio-temporal demand modeling [6].

Traffic profiles: We construct time-of-day multipliers for travel times (e.g., morning/evening peaks), optionally augmented by incident sampling [1][5].

C. Forecasting Models

We implement multiple forecasting baselines to separate *data quality* from *routing algorithm quality* [8]:

- **Demand forecasting:** seasonal ARIMA/Prophet-style baselines vs. LSTM/Temporal CNN (zone-level arrival counts) [7].
- **Traffic prediction:** regression on time-of-day + road class; optionally Graph Neural Networks for edge embeddings [9].
- **Decision-focused learning (optional):** SPO/SPO+ loss for travel-time prediction to improve routing outcomes directly [16][17].

D. Hybrid Routing and Algorithms

We compare three solver families, each run under the same simulator:

- 1) **Classical solver baseline:** OR-Tools VRP (time windows / capacities) and a deterministic time-dependent variant [7].
- 2) **Metaheuristics:** ALNS as a strong, widely used baseline [13], plus ACO/ABC and hybrid variants motivated by swarm intelligence [21][22].
- 3) **Learning-augmented methods:** RL-based route construction or online dispatch policies (benchmarking only; not assumed best) [10][9][11].

E. Online Execution and Rolling-Horizon Re-Optimization

To model reality, we adopt a rolling-horizon approach:

- Plan routes at time t using forecasts $\hat{\lambda}(t : t + H)$ and $\hat{\tau}(t : t + H)$.
- Execute for Δt , reveal new orders/traffic, then re-optimize with warm-start (route repair) [1][5].

This matches how real dispatch systems operate and allows us to measure *replanning cost* and *route stability*.

V. EXPERIMENTAL DESIGN

A. Benchmarked Scenarios

We evaluate multiple stress settings, inspired by SDVRP benchmarks:

- **Demand regimes:** low/medium/high arrival intensity; clustered vs. uniform spatial demand [11].
- **Traffic regimes:** mild vs. heavy congestion; incident probability on arterial edges [1].
- **Fleet regimes:** varying number of vehicles and capacities; optional time windows for service quality [15].

VI. RESULTS AND FINDINGS

A. Experimental Overview

Stochastic vehicle routing examples from the Chandigarh urban delivery scenario were used to assess each solver. Monte Carlo rollouts were used to simulate each instance in order to estimate realized cost, lateness, and variability. Optimality gap, computational efficiency, scalability, and robustness were used to compare performance.

1) Key Insights:

B. Mean Cost per Delivery vs Problem Size

We assess the mean cost per delivery as a function of problem size in order to examine the scalability of solution quality. To capture structural performance differences across scales, the analysis is divided into small-scale instances ($n < 1000$ customers) and large-scale instances ($n \geq 1000$ customers). Table I summarizes aggregated mean performance metrics in addition to the graphical analysis.



Fig. 2. Mean cost per delivery across solvers for small-scale instances ($n < 1000$ customers).

1) **Small-Scale Instances ($n < 1000$):** For instances ranging from 50 to 500 customers, Figure 2 and the numerical values presented in Table I demonstrate observable variability in solver performance. When compared to metaheuristic methods like ACO and Random, OR-Tools and Nearest Neighbor typically achieve lower mean cost per delivery. Performance rankings, however, vary depending on the size of the problem, suggesting that instance structure is important.

Important findings:

- Deterministic solvers consistently achieve competitive or lower delivery costs.
- Metaheuristic methods exhibit higher cost variability across instance sizes.
- No single solver strictly dominates across all small-scale configurations.
- Performance differences remain moderate but unstable in this range.

These results show that classical deterministic routing strategies are still very competitive for smaller urban delivery networks, as shown in Table I.

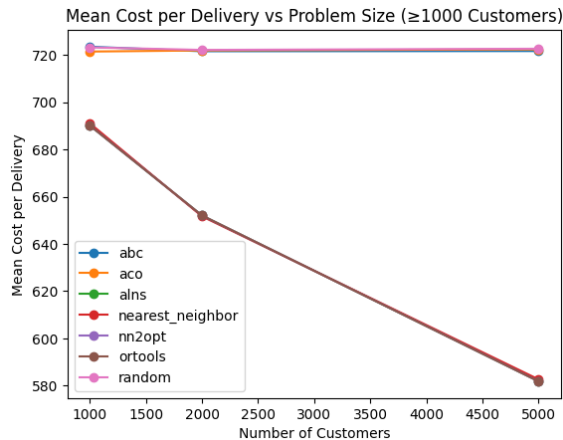


Fig. 3. Mean cost per delivery across solvers for large-scale instances ($n \geq 1000$ customers).

2) *Large-Scale Instances* ($n \geq 1000$): For large-scale instances (1000, 2000, and 5000 customers), solver behavior becomes more structured, as illustrated in Figure 3 and numerically summarized in Table I. Algorithm classes become more distinct from one another.

Important findings:

- OR-Tools and Nearest Neighbor exhibit stronger cost efficiency as scale increases.
- Metaheuristic solvers demonstrate cost stagnation at higher customer sizes.
- Performance ranking stabilizes at larger scales.
- Structural scalability differences become more pronounced beyond 1000 customers.

C. Computational Complexity and Scalability Analysis

Figure 4 summarizes the experimental results for solver performance across different instance sizes, and Table I provides numerical results. Two different growth patterns are revealed by a thorough examination of the scalability trends:

- **High Complexity Metaheuristics:** When we deal with a high number of customers, i.e., 1000 customers or more, we can observe a slowdown in both ALNS and OR-Tools. As we can see in Table I, both solvers take approximately 7.35 seconds each, which indicates that the complexity of both solvers' optimization processes is high due to the use of exhaustive search in local search strategies.
- **Inefficient Scaling in Greedy Heuristics:** Greedy methods like *nearest_neighbor* and *nn2opt* work well at first, but they don't work well with this dataset. Table I shows that their runtimes go up quickly from the millisecond range for small instances to the 5s range for larger instances. This suggests that their $O(n^2)$ or $O(n^3)$ complexity becomes a problem as the number of customers grows.

- **Algorithmic Stability:** On the other hand, *abc* and *aco* have consistent runtimes (about 2 seconds), as shown in Table I. This suggests that their performance is based on fixed parameters rather than the input size growing linearly or quadratically.
- **Baseline Efficiency:** The *random* solver maintains negligible overhead ($< 0.001s$), consistent with the values reported in Table I, confirming it as the lower bound for computational effort.

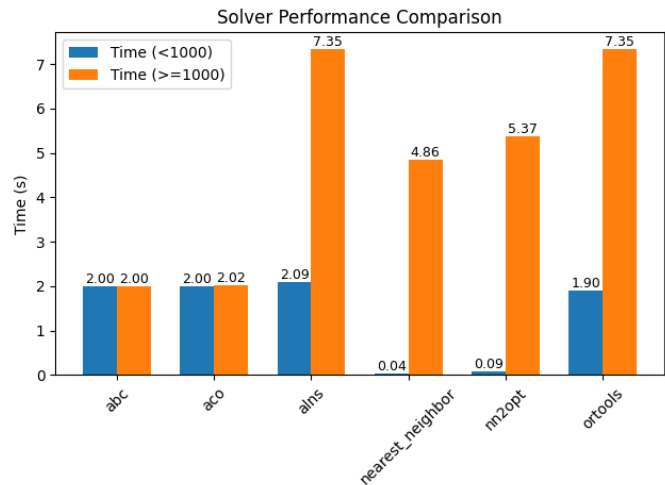


Fig. 4. Analysis of Solver Execution Times: Comparing Small-Scale ($n < 1000$) vs. Large-Scale ($n \geq 1000$) Performance.

D. Mean Lateness Analysis Across Instance Scales

Figure 5 and Table I Shows the average lateness (in seconds) for each solver in two groups: small-scale (less than 1,000 customers) and large-scale (1,000 or more customers).

a) *Scale Sensitivity.*: There is a difference in how sensitive solvers are to instance size. The ABC metaheuristic shows that the mean lateness goes down as the problem size goes up, as shown in Table I. This means that it is more stable when routing structures are denser. For larger cases, ACO and Nearest Neighbor also show moderate decreases in lateness, which suggests that they work better when there are more people in the same area.

In contrast, ALNS, NN2OPT, and OR-Tools show slight increases in mean lateness for $n \geq 1000$, consistent with the aggregated results in Table I. This behavior is caused by a lot of small bad decisions or more random changes in bigger networks. The random baseline stays pretty stable across scales, but it always does worse than structured approaches.

b) *Relative Ranking Stability.*: The relative ranking of solvers stays mostly the same across both instance categories, as shown in Table I. Metaheuristic methods like ABC, ALNS, and ACO work well in a competitive setting, while greedy-based methods work moderately well.

c) *Scalability Implications.*: The differences between small and large instances are not very big; they are moderate. This shows that lateness grows at a slower rate than customer

TABLE I
PERFORMANCE COMPARISON OF SOLVERS FOR DIFFERENT CUSTOMER SCALES

Solver	<1000 Customers			≥1000 Customers		
	Mean Time (s)	Mean Cost	Mean Lateness (s)	Mean Time (s)	Mean Cost	Mean Lateness (s)
abc	2.0003	723.44	2657.68	2.0025	722.31	2476.58
aco	2.0024	726.96	2432.47	2.0225	721.97	2347.16
alns	2.0887	715.81	2242.31	7.3481	641.36	2462.07
nearest_neighbor	0.0423	715.15	2278.41	4.8565	641.82	2462.35
nn2opt	0.0874	715.81	2242.31	5.3685	641.36	2462.07
ortools	1.8980	712.53	2436.30	7.3483	641.36	2462.07
random	0.0001	725.22	2465.99	0.0008	722.72	2410.56

size, which means that most algorithms should be able to handle larger numbers of customers. No solver shows a huge drop in lateness when demand goes up.

d) *Operational Interpretation.*: From a practical standpoint, ABC and ACO seem to be more effective at managing service delays on a large scale, as shown in Table I. When used on bigger networks, deterministic local improvement heuristics like NN2OPT might work better with better diversification mechanisms.

e) *Conclusion.*: The results indicate that algorithmic architecture predominates scale effects in influencing mean lateness performance. There are some small size-dependent differences, but the fact that there hasn't been a major performance collapse shows that the solver's behavior stays the same even when customer demand rises.

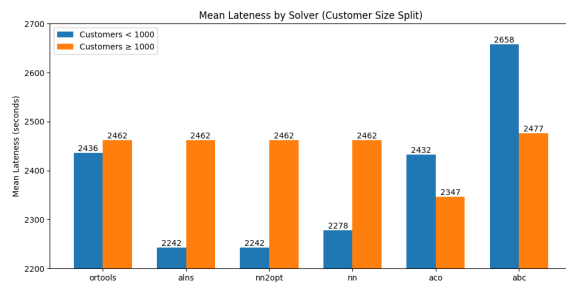


Fig. 5. Mean lateness (seconds) across solvers for small ($n < 1000$) and large ($n \geq 1000$) customer instances.

E. Cost per Delivery vs Lateness

Figure 6 and the combined metrics in Table I show how the average cost per delivery and the average lateness are related for the solvers that were tested. There is a clear positive link: solvers who have lower costs per delivery also tend to be less late.

Improvement-based heuristics like *nn2opt* and *ALNS* make up the largest group. They have both low delivery costs and very little lateness, as shown in Table I. The nearest neighbor heuristic performs competitively, with slightly higher lateness but similar cost efficiency.

On the other hand, *ABC*, *ACO*, and the random baseline have much higher delivery costs and are late more often, which is in line with the numbers in Table I. There is no valid relationship between delivery cost and service levels; while

lower-cost solvers intensively deliver the effective timeless performance.

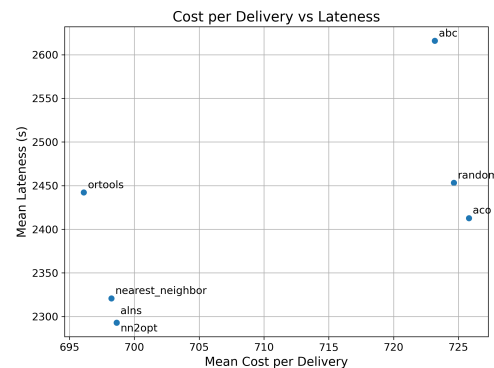


Fig. 6. Relationship between mean cost per delivery and mean lateness across solvers. Low-cost solvers (*nn2opt*, *ALNS*, *nearest_neighbor*) exhibit reduced lateness, indicating structural solution quality rather than a cost–service tradeoff.

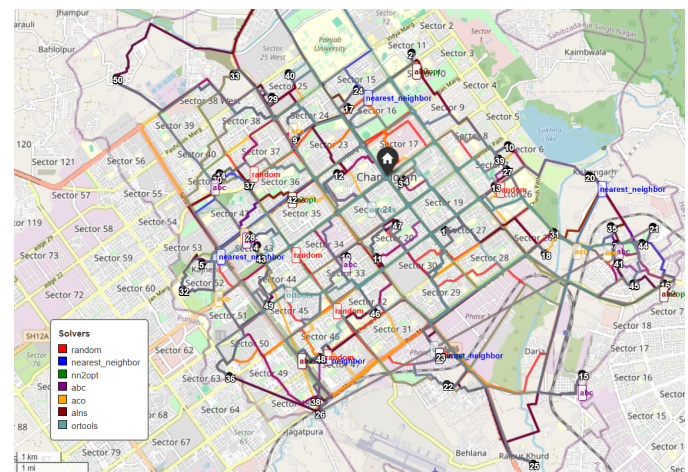


Fig. 7. Geographic visualization of routing solutions generated by multiple VRP solvers for a Chandigarh delivery instance. Customer nodes are derived from OpenStreetMap data, and colored paths represent routes produced by different algorithms.

Figure 7 presents the geographic visualization of the routes generated by multiple VRP solvers for a Chandigarh-based delivery instance of size 50 customers. Customer locations derived from OpenStreetMap nodes are represented as markers. The colored polylines indicate the routes that were con-

structed by different algorithms. The depot is represented as the central dispatch location from which all routes originate and terminate. The routes follow realistic road-network paths, allowing qualitative comparison of solver behaviors such as, path overlap, and spatial distribution of deliveries.

VII. CONCLUSION

This study evaluated multiple vehicle routing solvers under stochastic demand conditions using cost, lateness, and computation time as performance metrics.

The results show that OR-Tools achieves the lowest mean cost per delivery, providing a 4.09% improvement over the worst-performing solver (ACO). In terms of service quality, ALNS achieves the lowest mean lateness, reducing lateness by 12.34% compared to the highest-lateness solver (ABC).

A moderate positive correlation was found between the average cost per delivery and the average lateness ($r = 0.667$), indicating that cheaper solutions go hand in hand with better service performance. On the flip side, computation time had almost no correlation with lateness ($r = -0.063$), indicating that smarter solutions do not necessarily mean longer computation times.

From a computational perspective, the nearest neighbor heuristic was found to be the fastest optimized approach, with computation times being approximately 65% faster than the ALNS approach while maintaining competitive cost and lateness performance. To conclude, improvement-based heuristics such as the 2-opt and OR-tools provide a structurally better routing solution compared to swarm and random approaches, particularly when balancing cost and service reliability is a priority.

ACKNOWLEDGMENT

We are grateful to Dr. Gurwinder Singh, their research guide, for his valuable guidance, support, and encouragement during this research study. We also thank our friends and colleagues for their valuable feedback and support. Lastly, we are grateful to Chandigarh University for providing the necessary resources and a conducive environment to carry out this research.

REFERENCES

- [1] S. Heakl, S. Sukhbaatar, K.-H. Wang, K. Talebghasem, H. Najjran, and A. Abolghasemi, "SVRPBench: A Stochastic Vehicle Routing Problems Benchmark and Library for Evaluating Robustness under Real-World Stochastic Dynamics," arXiv:2505.21887, 2025.
- [2] G. B. Dantzig and J. H. Ramser, "The Truck Dispatching Problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
- [3] G. Laporte, "The Vehicle Routing Problem: An Overview of Exact and Approximate Algorithms," *European Journal of Operational Research*, 1992.
- [4] H. N. Psaraftis, M. Wen, and C. A. Kontovas, "Dynamic Vehicle Routing Problems: Three Decades and Counting," *Networks*, 2016.
- [5] N. Mardešić, T. Erdelić, T. Carić, and M. Đurasević, "Review of Stochastic Dynamic Vehicle Routing in the Evolving Urban Logistics Environment," *Mathematics*, vol. 12, no. 1, p. 28, 2024.
- [6] N. Giuffrida, J. Fajardo-Calderin, A. D. Masegosa, F. Werner, M. Steudter, and F. Pilla, "Optimization and Machine Learning Applied to Last-Mile Logistics: A Review," *Sustainability*, vol. 14, p. 5329, 2022.
- [7] R. Shahbazian *et al.*, "Integrating Machine Learning Into Vehicle Routing Problem Methods and Applications," *IEEE Access*, vol. 12, pp. 103796–103839, 2024.
- [8] A. Boggyrbayeva, A. Wilms, P. Friesen, S. Atabayev, and A. Sadeghi-Barzani, "Machine Learning to Solve Vehicle Routing Problems: A Survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 6, pp. 5377–5397, Jun. 2024.
- [9] J. J. Q. Yu, W. Yu, and J. Gu, "Online Vehicle Routing With Neural Combinatorial Optimization and Deep Reinforcement Learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 10, pp. 3806–3817, Oct. 2019.
- [10] M. Nazari, A. Oroojlooy, L. Snyder, and M. Takáč, "Reinforcement Learning for Solving the Vehicle Routing Problem," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [11] A. K. Kalakanti, S. Verma, T. Paul, and T. Yoshida, "RL SolVeR Pro: Reinforcement Learning for Solving Vehicle Routing Problem," in *Proc. 1st Int. Conf. on Artificial Intelligence and Data Sciences (AiDAS)*, 2019.
- [12] G. Boeing, "OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks," *Computers, Environment and Urban Systems*, 2017.
- [13] D. Pisinger and S. Ropke, "A General Heuristic for Vehicle Routing Problems," *Computers & Operations Research*, 2007.
- [14] N. Mladenović and P. Hansen, "Variable Neighborhood Search," *Computers & Operations Research*, 1997.
- [15] M. M. Solomon, "Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints," *Operations Research*, 1987.
- [16] A. N. Elmachtoub and P. Grigas, "Smart Predict, Then Optimize," arXiv:1710.08005 (v5), 2020.
- [17] H. Chu, W. Zhang, P. Bai, and Y. Chen, "Data-driven Optimization for Last-mile Delivery," *Complex & Intelligent Systems*, vol. 9, pp. 2271–2284, 2023 (online 2021).
- [18] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski, *Robust Optimization*. Princeton University Press, 2009.
- [19] A. Shapiro, D. Dentcheva, and A. Ruszczyński, *Lectures on Stochastic Programming: Modeling and Theory*. SIAM, 2014.
- [20] J. C. Ferreira and M. Esperança, "(Title as in source) Integration of AI and Electric Vehicles for Sustainable Last-Mile Delivery (Lisbon case study)," *World Electric Vehicle Journal*, vol. 16, p. 242, 2025.
- [21] R. Singh and G. Singh, "Hybrid approach for Drone-Based Delivery: Artificial Bee Colony with Constraint Handling (Chandigarh case study)," (IEEE-format paper provided as reference), 2025.
- [22] Y. Yan, J. Zhang, and Y. Wang, "A Route Optimization Model for Unmanned Aerial Vehicles in Package Delivery," (Proceedings/venue as in source), 2019.
- [23] P. Toth and D. Vigo, *Vehicle Routing: Problems, Methods, and Applications*. SIAM, 2014.
- [24] G. Clarke and J. W. Wright, "Scheduling of Vehicles from a Central Depot to a Number of Delivery Points," *Operations Research*, 1964.